

Problem1

1.

(1)method A(DCGAN)

Generator:

```

DC_Generator(
  (main): Sequential(
    (0): ConvTranspose2d(100, 512, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): ConvTranspose2d(512, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (4): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU(inplace=True)
    (6): ConvTranspose2d(256, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (7): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (8): ReLU(inplace=True)
    (9): ConvTranspose2d(128, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (10): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (11): ReLU(inplace=True)
    (12): ConvTranspose2d(64, 3, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (13): Tanh()
  )
)

```

Discriminator:

```

DC_Discriminator(
  (main): Sequential(
    (0): Conv2d(3, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (1): LeakyReLU(negative_slope=0.2, inplace=True)
    (2): Conv2d(64, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (3): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (4): LeakyReLU(negative_slope=0.2, inplace=True)
    (5): Conv2d(128, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (6): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (7): LeakyReLU(negative_slope=0.2, inplace=True)
    (8): Conv2d(256, 512, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (9): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): LeakyReLU(negative_slope=0.2, inplace=True)
    (11): Conv2d(512, 1, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (12): Sigmoid()
  )
)

```

(2)method B(SNGAN)

Generator:

```

SNGAN_Generator(
  (dense): Linear(in_features=100, out_features=16384, bias=True)
  (final): Conv2d(64, 3, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (model): Sequential(
    (0): ResBlockGenerator(
      (conv1): Conv2d(1024, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (model): Sequential(
        (0): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (1): LeakyReLU(negative_slope=0.2)
        (2): Upsample(scale_factor=2.0, mode=nearest)
        (3): Conv2d(1024, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (5): LeakyReLU(negative_slope=0.2)
        (6): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      )
      (bypass_conv): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1))
      (bypass): Sequential(
        (0): Upsample(scale_factor=2.0, mode=nearest)
        (1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1))
      )
    )
    (1): ResBlockGenerator(
      (conv1): Conv2d(512, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (model): Sequential(
        (0): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (1): LeakyReLU(negative_slope=0.2)
        (2): Upsample(scale_factor=2.0, mode=nearest)
        (3): Conv2d(512, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (5): LeakyReLU(negative_slope=0.2)
        (6): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      )
      (bypass_conv): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1))
      (bypass): Sequential(
        (0): Upsample(scale_factor=2.0, mode=nearest)
        (1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1))
      )
    )
    (2): ResBlockGenerator(
      (conv1): Conv2d(256, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (model): Sequential(
        (0): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (1): LeakyReLU(negative_slope=0.2)
        (2): Upsample(scale_factor=2.0, mode=nearest)
        (3): Conv2d(256, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (5): LeakyReLU(negative_slope=0.2)
        (6): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      )
      (bypass_conv): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1))
      (bypass): Sequential(
        (0): Upsample(scale_factor=2.0, mode=nearest)
        (1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1))
      )
    )
  )
)

```

```

(3): ResBlockGenerator(
  (conv1): Conv2d(128, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (model): Sequential(
    (0): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): Upsample(scale_factor=2.0, mode=nearest)
    (3): Conv2d(128, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): LeakyReLU(negative_slope=0.2)
    (6): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  )
  (bypass_conv): Conv2d(128, 64, kernel_size=(1, 1), stride=(1, 1))
  (bypass): Sequential(
    (0): Upsample(scale_factor=2.0, mode=nearest)
    (1): Conv2d(128, 64, kernel_size=(1, 1), stride=(1, 1))
  )
)
(4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(5): LeakyReLU(negative_slope=0.2)
(6): Conv2d(64, 3, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(7): Tanh()
)
)

```

Discriminator:

```
SNGAN_Discriminator(
  (model): Sequential(
    (0): FirstResBlockDiscriminator(
      (conv1): ParametrizedConv2d(
        3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
        (parametrizations): ModuleDict(
          (weight): ParametrizationList(
            (0): _SpectralNorm()
          )
        )
      )
      (conv2): ParametrizedConv2d(
        64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
        (parametrizations): ModuleDict(
          (weight): ParametrizationList(
            (0): _SpectralNorm()
          )
        )
      )
      (bypass_conv): ParametrizedConv2d(
        3, 64, kernel_size=(1, 1), stride=(1, 1)
        (parametrizations): ModuleDict(
          (weight): ParametrizationList(
            (0): _SpectralNorm()
          )
        )
      )
    )
    (model): Sequential(
      (0): ParametrizedConv2d(
        3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
        (parametrizations): ModuleDict(
          (weight): ParametrizationList(
            (0): _SpectralNorm()
          )
        )
      )
      (1): LeakyReLU(negative_slope=0.2)
      (2): ParametrizedConv2d(
        64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
        (parametrizations): ModuleDict(
          (weight): ParametrizationList(
            (0): _SpectralNorm()
          )
        )
      )
      (3): AvgPool2d(kernel_size=2, stride=2, padding=0)
    )
    (bypass): Sequential(
      (0): AvgPool2d(kernel_size=2, stride=2, padding=0)
      (1): ParametrizedConv2d(
        3, 64, kernel_size=(1, 1), stride=(1, 1)
        (parametrizations): ModuleDict(
          (weight): ParametrizationList(
            (0): _SpectralNorm()
          )
        )
      )
    )
  )
)
```

```

(1): ResBlockDiscriminator(
  (conv1): ParametrizedConv2d(
    64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (conv2): ParametrizedConv2d(
    128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (model): Sequential(
    (0): LeakyReLU(negative_slope=0.2)
    (1): ParametrizedConv2d(
      64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
      (parametrizations): ModuleDict(
        (weight): ParametrizationList(
          (0): _SpectralNorm()
        )
      )
    )
    (2): LeakyReLU(negative_slope=0.2)
    (3): ParametrizedConv2d(
      128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
      (parametrizations): ModuleDict(
        (weight): ParametrizationList(
          (0): _SpectralNorm()
        )
      )
    )
    (4): AvgPool2d(kernel_size=2, stride=2, padding=0)
  )
  (bypass_conv): ParametrizedConv2d(
    64, 128, kernel_size=(1, 1), stride=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (bypass): Sequential(
    (0): ParametrizedConv2d(
      64, 128, kernel_size=(1, 1), stride=(1, 1)
      (parametrizations): ModuleDict(
        (weight): ParametrizationList(
          (0): _SpectralNorm()
        )
      )
    )
    (1): AvgPool2d(kernel_size=2, stride=2, padding=0)
  )
)

```

```

(2): ResBlockDiscriminator(
  (conv1): ParametrizedConv2d(
    128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (conv2): ParametrizedConv2d(
    256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
)
(model): Sequential(
  (0): LeakyReLU(negative_slope=0.2)
  (1): ParametrizedConv2d(
    128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (2): LeakyReLU(negative_slope=0.2)
  (3): ParametrizedConv2d(
    256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (4): AvgPool2d(kernel_size=2, stride=2, padding=0)
)
(bypass_conv): ParametrizedConv2d(
  128, 256, kernel_size=(1, 1), stride=(1, 1)
  (parametrizations): ModuleDict(
    (weight): ParametrizationList(
      (0): _SpectralNorm()
    )
  )
)
(bypass): Sequential(
  (0): ParametrizedConv2d(
    128, 256, kernel_size=(1, 1), stride=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (1): AvgPool2d(kernel_size=2, stride=2, padding=0)
)
)

```

```

(3): ResBlockDiscriminator(
  (conv1): ParametrizedConv2d(
    256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (conv2): ParametrizedConv2d(
    512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
)
(model): Sequential(
  (0): LeakyReLU(negative_slope=0.2)
  (1): ParametrizedConv2d(
    256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (2): LeakyReLU(negative_slope=0.2)
  (3): ParametrizedConv2d(
    512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (4): AvgPool2d(kernel_size=2, stride=2, padding=0)
)
(bypass_conv): ParametrizedConv2d(
  256, 512, kernel_size=(1, 1), stride=(1, 1)
  (parametrizations): ModuleDict(
    (weight): ParametrizationList(
      (0): _SpectralNorm()
    )
  )
)
(bypass): Sequential(
  (0): ParametrizedConv2d(
    256, 512, kernel_size=(1, 1), stride=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (1): AvgPool2d(kernel_size=2, stride=2, padding=0)
)
)

```

```

(4): ResBlockDiscriminator(
  (conv1): ParametrizedConv2d(
    512, 1024, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (conv2): ParametrizedConv2d(
    1024, 1024, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (model): Sequential(
    (0): LeakyReLU(negative_slope=0.2)
    (1): ParametrizedConv2d(
      512, 1024, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
      (parametrizations): ModuleDict(
        (weight): ParametrizationList(
          (0): _SpectralNorm()
        )
      )
    )
    (2): LeakyReLU(negative_slope=0.2)
    (3): ParametrizedConv2d(
      1024, 1024, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)
      (parametrizations): ModuleDict(
        (weight): ParametrizationList(
          (0): _SpectralNorm()
        )
      )
    )
  )
  (bypass_conv): ParametrizedConv2d(
    512, 1024, kernel_size=(1, 1), stride=(1, 1)
    (parametrizations): ModuleDict(
      (weight): ParametrizationList(
        (0): _SpectralNorm()
      )
    )
  )
  (bypass): Sequential(
    (0): ParametrizedConv2d(
      512, 1024, kernel_size=(1, 1), stride=(1, 1)
      (parametrizations): ModuleDict(
        (weight): ParametrizationList(
          (0): _SpectralNorm()
        )
      )
    )
  )
  (5): LeakyReLU(negative_slope=0.2)
)
(fc): ParametrizedLinear(
  in_features=1024, out_features=1, bias=True
  (parametrizations): ModuleDict(
    (weight): ParametrizationList(
      (0): _SpectralNorm()
    )
  )
)

```


2.

methodA(DCGAN)



methodB(SNGAN)



We can see that SNGAN improves the eyes and the overall color is more similar to train dataset.

3.

It is hard to deal with mode collapse. I encountered mode collapse when I trained SNGAN, so I searched many articles on the Internet and tried many methods to solve it. Finally I set the value of true_label to 0.9. This will let the discriminator not to do the job too good to prevent the generator go into mode collapse in order to pass discriminator.

Problem2

1.

```
DDPM(
  (nn_model): ContextUnet(
    (init_conv): ResidualConvBlock(
      (conv1): Sequential(
        (0): Conv2d(3, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
      (conv2): Sequential(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
    )
    (down1): UnetDown(
      (model): Sequential(
        (0): ResidualConvBlock(
          (conv1): Sequential(
            (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
            (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): GELU(approximate=none)
          )
          (conv2): Sequential(
            (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
            (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): GELU(approximate=none)
          )
        )
        (1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
      )
    )
    (down2): UnetDown(
      (model): Sequential(
        (0): ResidualConvBlock(
          (conv1): Sequential(
            (0): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
            (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): GELU(approximate=none)
          )
          (conv2): Sequential(
            (0): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
            (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): GELU(approximate=none)
          )
        )
        (1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
      )
    )
    (to_vec): Sequential(
      (0): AvgPool2d(kernel_size=7, stride=7, padding=0)
      (1): GELU(approximate=none)
    )
  )
)
```

```

(timeembed1): EmbedFC(
  (model): Sequential(
    (0): Linear(in_features=1, out_features=512, bias=True)
    (1): GELU(approximate=none)
    (2): Linear(in_features=512, out_features=512, bias=True)
  )
)
(timeembed2): EmbedFC(
  (model): Sequential(
    (0): Linear(in_features=1, out_features=256, bias=True)
    (1): GELU(approximate=none)
    (2): Linear(in_features=256, out_features=256, bias=True)
  )
)
(contextembed1): EmbedFC(
  (model): Sequential(
    (0): Linear(in_features=10, out_features=512, bias=True)
    (1): GELU(approximate=none)
    (2): Linear(in_features=512, out_features=512, bias=True)
  )
)
(contextembed2): EmbedFC(
  (model): Sequential(
    (0): Linear(in_features=10, out_features=256, bias=True)
    (1): GELU(approximate=none)
    (2): Linear(in_features=256, out_features=256, bias=True)
  )
)
(up0): Sequential(
  (0): ConvTranspose2d(512, 512, kernel_size=(7, 7), stride=(7, 7))
  (1): GroupNorm(8, 512, eps=1e-05, affine=True)
  (2): ReLU()
)

```

```

(up1): UnetUp(
  (model): Sequential(
    (0): ConvTranspose2d(1024, 256, kernel_size=(2, 2), stride=(2, 2))
    (1): ResidualConvBlock(
      (conv1): Sequential(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
      (conv2): Sequential(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
    )
    (2): ResidualConvBlock(
      (conv1): Sequential(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
      (conv2): Sequential(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
    )
  )
)

```

```

(up2): UnetUp(
  (model): Sequential(
    (0): ConvTranspose2d(512, 256, kernel_size=(2, 2), stride=(2, 2))
    (1): ResidualConvBlock(
      (conv1): Sequential(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
      (conv2): Sequential(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
    )
    (2): ResidualConvBlock(
      (conv1): Sequential(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
      (conv2): Sequential(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): GELU(approximate=none)
      )
    )
  )
)
(out): Sequential(
  (0): Conv2d(512, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): GroupNorm(8, 256, eps=1e-05, affine=True)
  (2): ReLU()
  (3): Conv2d(256, 3, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
)
(loss_mse): MSELoss()

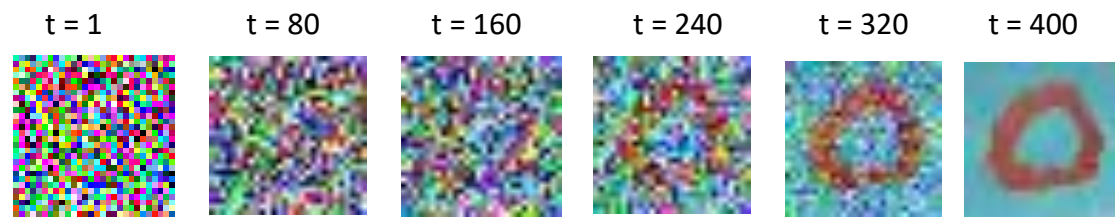
```

Since we need to give the DDPM the label, so we have to revise the original model of DDPM, the method I used is to create a context_mask in the middle of model. The mask is to let the forward process in the model to create the specific context in the final output.

2.



3.



4.

The conditional diffusion model consumes much time and memory when generating images. It will consume much time if you want to set a larger t in the model and consume much memory if you want to generate images at once time. But the training time is relatively less than generating. I think that the reason is that the training algorithm doesn't use the for loop but generating algorithm does.

Problem3

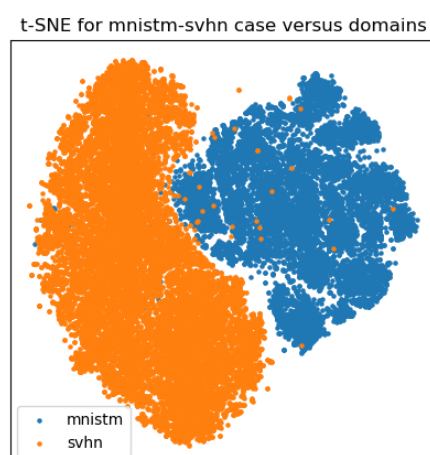
1.

	MNIST-M $\rightarrow$ SVHN	MNIST-M $\rightarrow$ USPS
Trained on source	0.39782	0.77201
Adaptation(DANN)	0.45439	0.81758
Trained on target	0.93022	0.99284

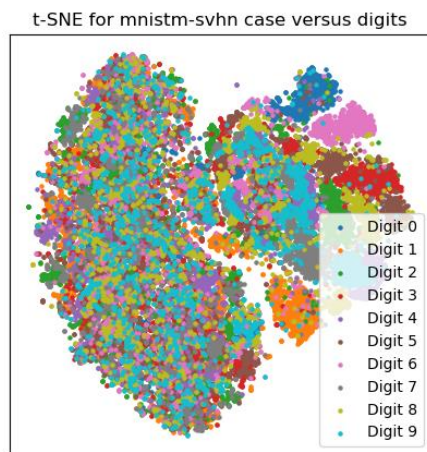
2.

MNIST-M $\rightarrow$ SVHN

(a)by domain

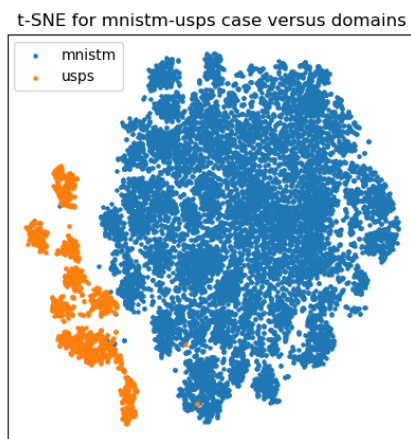


(b)by class

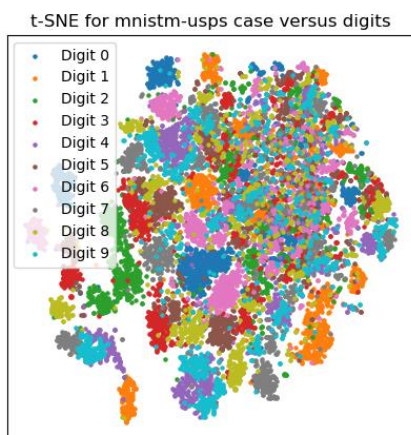


MNIST-M $\rightarrow$ USPS

(a) by domain



(b) by class



3.

The implementations of DANN is like the description in the paper. Since the size of source domain and target domain is different, so you may need to train more epochs when the source domain is larger to get a sufficient training steps. And I give the loss of domain a bias to let the model can learn quickly.

Reference:

1. <https://github.com/christiancosgrove/pytorch-spectral-normalization-gan>
2. https://github.com/TeaPearce/Conditional_Diffusion_MNIST